Lucera Trading Systems Challenge

HackRU Fall 2026 — October 10–11, 2026
Sponsored by Lucera Financial Infrastructures

> **Build a simple, reliable, and scalable system for modern electronic trading.**

Participant-facing challenge

Electronic trading depends on much more than a user interface. Behind every order or market update are systems that must communicate quickly, preserve correctness, handle concurrent activity, and remain understandable as they scale.

Build a working system that solves a meaningful problem in trading or financial-market infrastructure. The challenge is intentionally open-ended: choose the problem, architecture, languages, and tools that best demonstrate your engineering judgment.

We value:

- a small, clear design with well-defined responsibilities;
- sound engineering intuition in how the problem is scoped, modeled, tested, and explained;
- explicit correctness guarantees and evidence that they hold;
- thoughtful measurement under known resource limits; and
- an honest explanation of tradeoffs, limitations, and the next scaling bottleneck.

A project may be a system, developer tool, standalone library, or meaningful open-source contribution. A narrow implementation that works reliably and is measured carefully is stronger than a broad project that is mostly conceptual.

Possible project directions

These examples are inspiration, not required categories:

- a production-quality contribution to an open-source trading or messaging library;
- Python tooling or codec support for Simple Binary Encoding (SBE);
- a FIX message explainer that combines deterministic protocol interpretation with carefully bounded AI assistance;
- a real-time FIX log analyzer that reconstructs trading activity across large files or streams and produces a deterministic report;
- an API-driven backtesting engine with reproducible data, execution, and accounting behavior;
- an order book, matching engine, auction, or request-for-quote system;
- a paper-trading venue or prediction-market platform with dynamic markets;
- a market-data ingestion, normalization, distribution, or replay service;

- a low-latency messaging, event-streaming, or state-replication layer;
- pre-trade risk checks, position limits, exposure monitoring, or kill switches;
- an execution simulator, smart order router, or transaction-cost analyzer;
- a distributed ledger, audit trail, reconciliation service, or event-sourced system;
- a backtesting engine that prevents look-ahead bias and supports reproducible runs;
- exchange, broker, or client connectivity with realistic failure handling;
- observability, anomaly detection, or operational tooling for a trading system; or
- another infrastructure problem relevant to electronic markets.

For concrete starting points, suggested 24-hour scopes, correctness properties, and performance ideas, see the Lucera Project Starter Guide.

Projects do not need to execute real trades. Teams may use generated or historical data and free/read-only data sources such as yfinance or Alpaca Market Data, subject to the provider's terms.

Minimum track requirements

To be eligible for Lucera judging, a submission must:

1. **Address a trading or financial-market infrastructure use case.**
2. **Demonstrate a working implementation appropriate to the project type.**
    - **Systems and tools:** demonstrate a working end-to-end path. At least two components or clients must communicate as part of the core workflow. Components may be processes, services, nodes, producers/consumers, or user/system clients where the separation serves a real purpose.
    - **Libraries and open-source contributions:** demonstrate a substantial working capability integrated with the library's public interfaces. The contribution should be upstream-ready: follow the host project's contribution guidelines and conventions; include focused tests and documentation; preserve compatibility where required; and be presented as a reviewable patch, fork, or pull request. Acceptance or merge by the upstream maintainers is not required during the hackathon.
3. **State at least one important correctness property or invariant** and include automated tests or other repeatable evidence for it. Examples include price-time priority, conservation of positions or cash, deterministic replay, duplicate-event handling, ordering, idempotency, or enforcement of a risk limit.
4. **Provide reproducible quantitative evidence.** Measure at least one metric relevant to the system, such as latency, throughput, memory, CPU, startup/recovery time, replay speed, or concurrent-client capacity.
5. **Submit the project through HackRU's Devpost process**, including the source-code repository and all information required by HackRU.
6. **Include setup and run instructions**, a short architecture explanation, test instructions/results, and performance methodology/results.
7. **Disclose major external APIs, frameworks, libraries, datasets, and prebuilt components** and clearly identify what the team built during the hackathon.

All participants and submissions remain subject to the official HackRU rules, eligibility requirements, deadlines, and code of conduct.

Safety and data boundaries

- Simulated or paper execution is sufficient and encouraged.
- Public or properly authorized market-data APIs may be used in accordance with their terms.
- Do not include API secrets, account credentials, confidential information, unlawfully obtained data, or personal financial information in the submission.
- Do not place real orders or put real funds at risk for this challenge.
- Make clear where behavior is simulated and where external data is used.

Required submission evidence

The Devpost submission or linked repository should make it easy for a judge to find:

- a runnable demo of the core workflow;
- or, for a library contribution, a runnable example plus a reviewable patch or pull request;
- source code and dependency/setup instructions;
- an architecture diagram or concise component description;
- the correctness invariants being claimed;
- automated tests and their results;
- performance results from a confined environment; and
- known limitations and the next engineering steps.

Performance report template

Use a local machine, container, or VM with fixed limits. Projects need not use identical environments because their workloads may differ, but results must be transparent and reproducible.

Report:

1. **Environment:** CPU/vCPU count, memory limit, operating system, runtime versions, and relevant container/VM limits.
2. **Workload:** data size, event/order count, client count, test duration, and any important input distribution.
3. **Metric:** the result and why it matters to the project. For latency, include percentile measurements when practical rather than only an average.
4. **Procedure:** exact commands or scripts needed to repeat the measurement, including warm-up or setup.
5. **Interpretation:** the observed bottleneck, important tradeoffs, and what would likely fail or need redesign next.

Judges will evaluate the quality and relevance of the evidence, not rank unrelated projects by raw throughput.

Judging rubric

Judges score each category from 1 to 5 and apply the listed weight.

1. Correctness and reliability — 30%

- Are important invariants stated clearly and appropriate for the use case?

- Does the implementation preserve them under realistic activity and edge cases?
- Do tests provide meaningful, repeatable evidence?
- Are concurrency, ordering, duplication, invalid input, and failures handled where relevant?
- Does the team distinguish demonstrated behavior from assumptions or future work?

2. System design and simplicity — 25%

- Is the architecture easy to understand and appropriately scoped for 24 hours?
- Do components have clear responsibilities and interfaces?
- Does each distributed boundary have a defensible purpose?
- For a library contribution, does the change fit the host project's architecture, conventions, and public API?
- Are data models, protocols, and technology choices appropriate?
- Is there a credible scaling path without unnecessary complexity?

3. Performance and scalability evidence — 20%

- Is the chosen metric relevant to the system's intended use?
- Are resource limits, workload, methodology, and results disclosed?
- Can the measurement be reproduced?
- Does the team understand bottlenecks and performance/correctness tradeoffs?
- Is the design likely to scale in the dimensions the team claims?

4. Completeness and demonstration — 15%

- Does the core workflow run end to end?
- Do multiple components or clients interact successfully, or is the library contribution integrated and usable through its intended public interface?
- Are setup, tests, and the demo reliable and understandable?
- Does the implementation go beyond a mock-up, slide deck, or disconnected prototype?
- Are limitations handled honestly and documented clearly?

5. Trading relevance and domain understanding — 10%

- Does the project solve a meaningful electronic-trading or market-infrastructure problem?
- Are workflows, terminology, and assumptions internally consistent?
- Has the team identified relevant real-world constraints and edge cases?
- Does the system provide a useful foundation for a real trading application?

Scoring anchors

- **5 — Exceptional:** complete, convincing, and supported by strong technical evidence.
- **4 — Strong:** works well and shows sound judgment, with only minor gaps.
- **3 — Solid:** functioning submission with meaningful evidence but notable limitations.
- **2 — Partial:** some working elements, but major claims are unsupported or incomplete.
- **1 — Minimal:** mostly conceptual, nonfunctional, or missing essential evidence.

When scores are tied, judges should prefer the project with stronger end-to-end completeness, then stronger correctness evidence, then clearer technical communication. Visual polish alone should not

determine the result unless it is central to the system's function.

Judge review flow

1. **Eligibility check:** confirm that the minimum track requirements are present.
2. **Demo:** ask the team to run the core workflow and show component interaction, or demonstrate the contributed library capability through its public interface.
3. **Correctness:** ask which invariant matters most and how the tests attempt to break it.
4. **Performance:** inspect the confined-environment methodology and one reproducible result.
5. **Design:** ask why the architecture is this simple or complex and what reaches its limit first.
6. **Score:** rate each rubric category independently before calculating the weighted total.

Contact

Vijay Maddila — Lucera Financial Infrastructures

Participants may contact Vijay through the official HackRU/Devpost communication channel or speak with a Lucera representative at the event.

Short Devpost-ready version

Lucera Trading Systems Challenge

Build a simple, reliable, and scalable system for modern electronic trading.

Electronic markets rely on systems that move data, coordinate concurrent activity, enforce risk controls, preserve correctness, and recover from failure. Lucera invites you to build a working system that solves any meaningful problem in trading or financial-market infrastructure.

The challenge is intentionally open-ended. You might contribute Python support to an open-source SBE library, build an AI-assisted FIX explainer, create a deterministic real-time FIX report generator, design an API-driven backtesting engine, or build an order book, market-data pipeline, risk service, execution simulator, audit trail, exchange connector, or something we have not imagined. These are examples, not fixed categories.

Strong submissions will show engineering intuition through a simple, clear design; define and test an important correctness property; and provide at least one reproducible, resource-bounded performance measurement. Systems and tools should demonstrate a complete end-to-end workflow involving at least two communicating components or clients. Library and open-source projects should deliver a substantial, integrated, upstream-ready contribution with tests, documentation, compatibility, and a reviewable patch or pull request. Acceptance by the upstream maintainers is not required during the hackathon.

Your Devpost submission and linked repository must include:

- a working demo and source code;
- setup and run instructions;
- a concise architecture explanation;
- at least one stated invariant with automated tests or repeatable evidence;

- at least one relevant performance/resource metric, including environment limits, workload, procedure, and result; and
- disclosure of major external APIs, frameworks, datasets, and prebuilt components.

Generated or historical data and free/read-only sources such as yfinance or Alpaca Market Data are welcome. Keep execution simulated: do not place real orders, risk real funds, expose credentials, or use confidential or unlawfully obtained data.

Judging: correctness and reliability (30%); system design and simplicity (25%); performance and scalability evidence (20%); completeness and demonstration (15%); and trading relevance/domain understanding (10%).

All official HackRU rules, eligibility requirements, deadlines, and Devpost submission requirements apply.

Contact: Vijay Maddila, Lucera Financial Infrastructures, through the official HackRU/Devpost channel or at the event.